

# iRacing Coaching — Guia de Setup

## Opção A: Post-race Cloud MVP (COMECE AQUI)

### Instalação

```
bash

cd iracing_cloud_mvp
pip install -r requirements.txt
```

### Configuração de TTS gratuito

Não requer chave de API. O `edge-tts` usa as vozes neurais do Microsoft Edge. Teste rápido:

```
bash

python -c "import asyncio; import edge_tts; asyncio.run(edge_tts.Communicate('Freia
```

### Rodar a API

```
bash

cd iracing_cloud_mvp
uvicorn api.main:app --reload --port 8000
# Acesse: http://localhost:8000/docs
```

### Rodar o portal Streamlit

```
bash

streamlit run frontend/portal.py
# Acesse: http://localhost:8501
```

### Usar Claude API (opcional, substitui Ollama)

```
bash

export ANTHROPIC_API_KEY="sua-chave-aqui"
# No portal, selecione "claude-haiku" no dropdown de modelo
# Custo estimado: ~$0.002 por feedback completo
```

## TTS: comparativo de opções gratuitas

| Backend    | Qualidade     | Latência | Requer chave | Como ativar               |
|------------|---------------|----------|--------------|---------------------------|
| edge-tts   | ★★★★ (neural) | ~600ms   | Não          | padrão (TTS_BACKEND=edge) |
| gTTS       | ★★★           | ~800ms   | Não          | TTS_BACKEND=gtts          |
| ElevenLabs | ★★★★★         | ~400ms   | Sim (\$)     | TTS_BACKEND=elevenlabs    |

## Opção B: Real-time Daemon

### Pré-requisitos

- Windows (iRacing SDK é Windows-only)
- iRacing aberto e em pista
- VB-Audio Virtual Cable (gratuito): <https://vb-audio.com/Cable/> → Configure como dispositivo de saída padrão no Windows → Configure como entrada de microfone no iRacing

### Instalação

```
bash
cd iracing_realtime
pip install -r requirements.txt
```

### Rodar o daemon

```
bash

# Em um terminal separado, enquanto o iRacing está aberto:
python watcher.py --pilot tomaz --level intermediate --track spa

# O daemon vai:
# 1. Aguardar conexão com o iRacing
# 2. Detectar cada troca de setor
# 3. Gerar feedback via LLM + TTS
# 4. Tocar áudio pelo VB-Audio (injetado no iRacing como rádio)
```

### TODO para completar a Opção B

1. Importar `TRACK_CONFIGS` do seu `config.py` para as `sector_edges` reais
2. Integrar `analyze_snapshot` com o pipeline v3 para calcular `delta_time` vs referência

3. Calibrar o `num_predict=120` do LLM para o comprimento ideal de feedback oral

---

## Migração do Notebook para Módulo (ambas as opções precisam disso)

O próximo passo mais importante: extrair as funções do notebook v3 para um arquivo `pipeline/` importável.

```
pipeline/  
├─ __init__.py  
├─ llm_prompts.py      # get_system_prompt, build_user_prompt, SYSTEM_PROMPTS  
├─ heuristics.py       # build_tips_payload_by_sector, filter_tips_by_level  
├─ telemetry.py        # load_ibt, basic_clean_and_units, assign_sectors  
└─ stochastic.py       # compute_stochastic_profile
```

Isso desbloqueia as duas opções sem duplicar código.